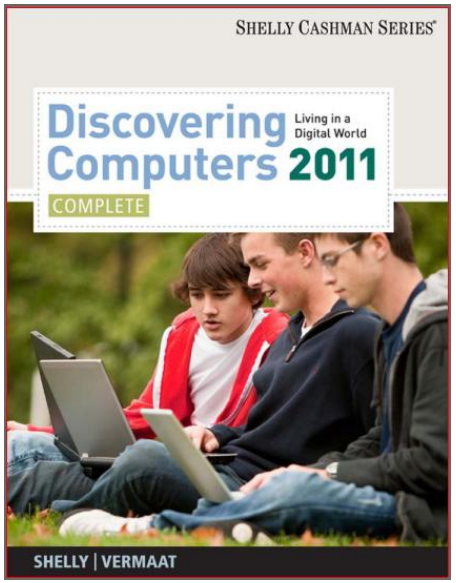
Lecture 2:



Real time software design methods

Topic to be Covered

Software Development for instrumentation:

Software development for Temperature (Low and High) level, Speed, pressure etc.

Embedded Software Development:

Assemblers, linkers and loaders. Binary file formats for processor executable files. Typical structure of timer-interrupt driven programs. GNU-GCC compiler introduction, programming with Linux environment and gnu debugging, gnu insight with step level trace debugging, make file interaction, building and execution.

Real time software design methods

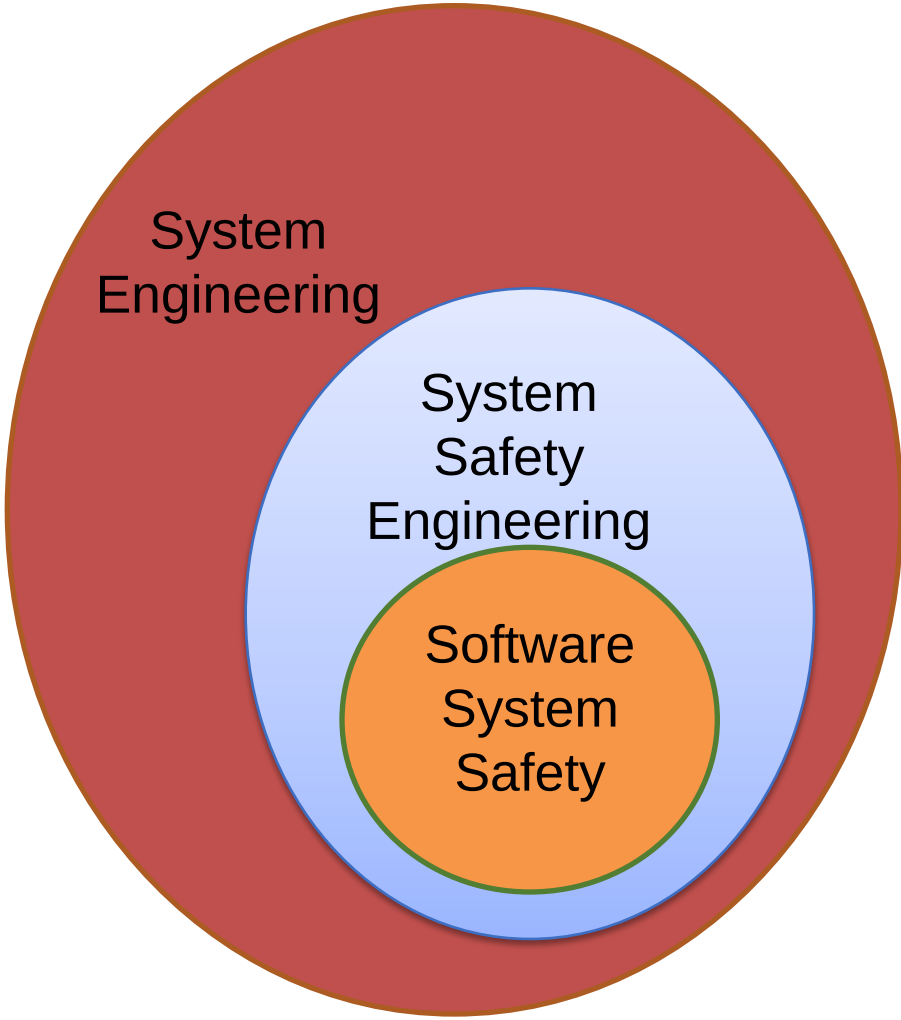
❖ Software Safety

- Software safety (software system safety) is the software engineering aspect that seeks to ensure active measures are taken to assure system integrity through prevention/ elimination, reduction and/or control of hazards that may be caused or induced by software

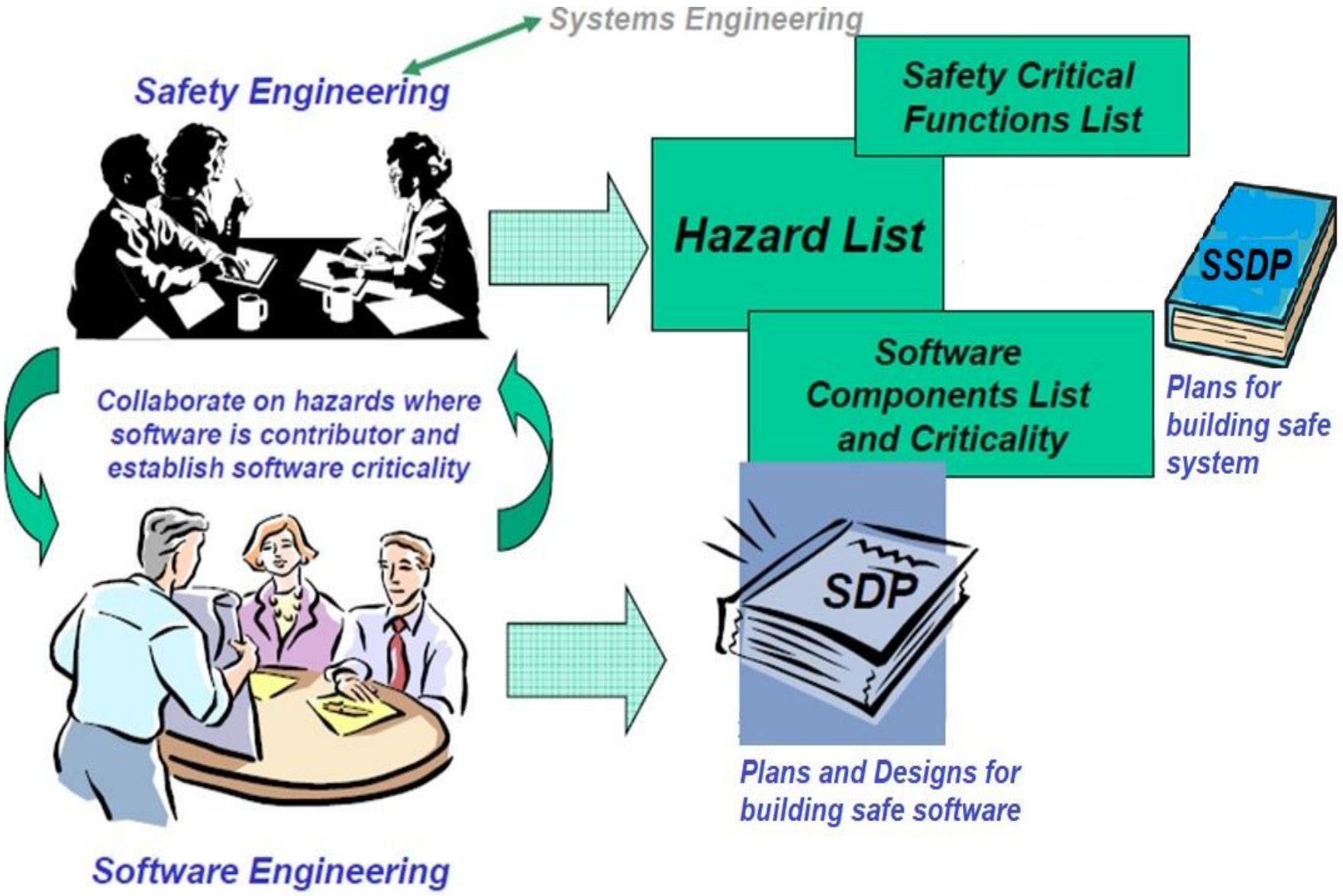
- Hazards are potential source of harm, to people, the environment or damage to property

Software Safety

- Software is a subset of system safety and system engineering
  - System engineering encompasses planning, identifying, documenting and mitigating hazards that contribute to mishaps/accidents
  - Systems engineering is an interdisciplinary field of engineering and engineering management that focuses on how to design, integrate and manage complex systems over their life cycles



Software and Safety Engineering



Safety-critical Software Systems

- Safety-critical software - software unit, component, or object whose proper recognition, control, performance, or fault tolerance is essential to the safe operation and support of the system in which it executes

- Safety-critical systems - systems whose failure could result in loss of life, significant property damage or damage to the environment

- Safety-critical functions - any function or integrated functions implemented in software that contributes to, commands, controls, or monitors system level safety-critical requirements needed to safely operate or support the system in which it execute

Group Activities

Consider any software that you might have been involved in its development or just are familiar with:

- Identify any key processing present in software
- Is there any potential hazard that may likely be caused by the software during the course of processing?
- What can we say about its safety criticality?

Software Safety and Safety Critical Systems

- **Software safety is traditionally considered in context of an operational (functional) system:**
  - **Computer operated drone/flight system (fly/drive by wire system)**
  - **Dynamic traffic light system**
  - **Heart pacemaker**
  - **Insulin pump**
  - **Auto/aircraft anti-lock brakes**

- **All have critical software processing that commands, controls, and/or monitors critical functions necessary for continued safe operation of that system**

Examples of Safety Critical Systems

- Traditional safety-critical systems
  - Aircraft/Drones, cars, weapons systems, medical devices and nuclear power plants
- Tool based safety critical systems
  - Code libraries, compilers, simulators, test rigs, requirements traceability tools
  - Other tools used to manage and build the kinds of systems we traditionally think of as safety-critical systems
- Software that provides data used to make decisions in safety-critical systems is likely safety-critical
  - An app that calculates how much fuel to load onto a commercial jet is safety critical because an incorrect result from that app may lead to a life threatening situation

Examples of Safety Critical Systems

- Other software running on the same system as safety-critical software may also be classified as safety-critical:
  - If it might adversely affect the safety functions of the safety-critical system.
    - E.g., by:
      - Blocking I/O, interrupts, or CPU
      - Using up RAM
      - Overwriting memory,
      - etc.

Typical Characteristics of a Safety Critical Software System

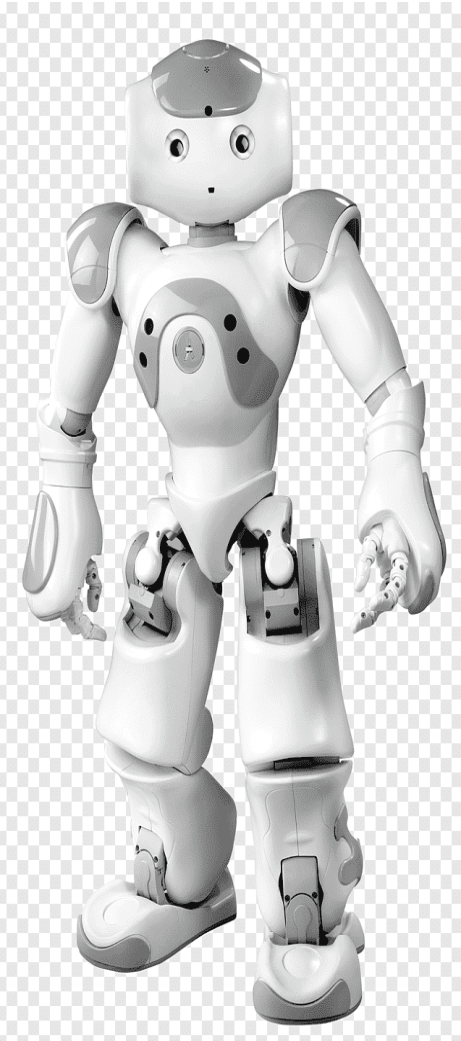
- Embedded, distributed systems
- Monitors its environment
  - With various kinds of sensors
  - Manipulates that environment by controlling actuators such as valves, hydraulics, solenoids, motors, etc. with or without the assistance of human operators

Group Activities

- Identify critical software processing present in each of the following operational systems:
  - **Computer operated drone/flight system**
  - **Computer operated Fire alarm system**
  - **Dynamic Traffic light control**
  - **Insulin pump**

Functional Safety

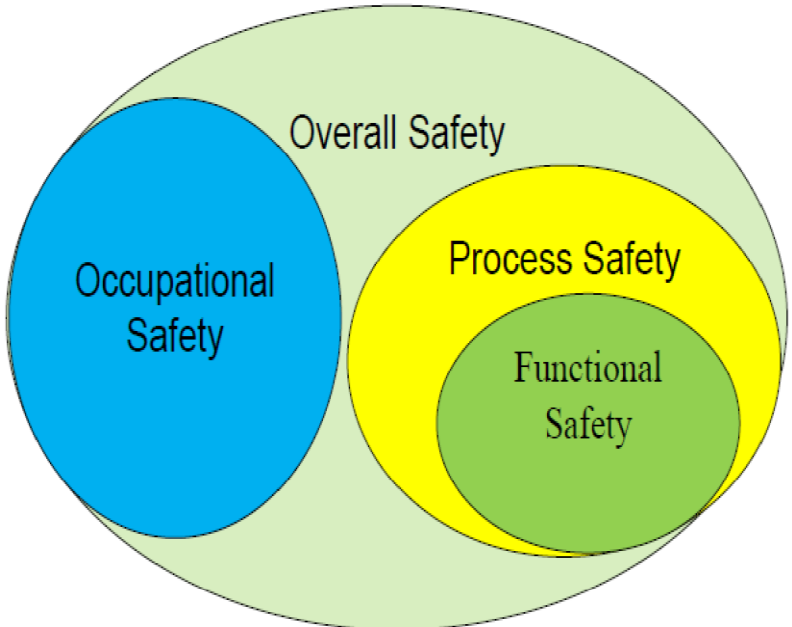
- Functional safety (FS) is to use functions (Safety functions) to reduce the risk of equipment causing harm to people, damage to property or society due to malfunction or incorrect operation (mishap)
  - Software system safety is synonymous with the software engineering aspects of Functional Safety
- An example of a functional safety feature is using motor control devices on robots to avoid hazards by automatically stopping the motor



Functional Safety

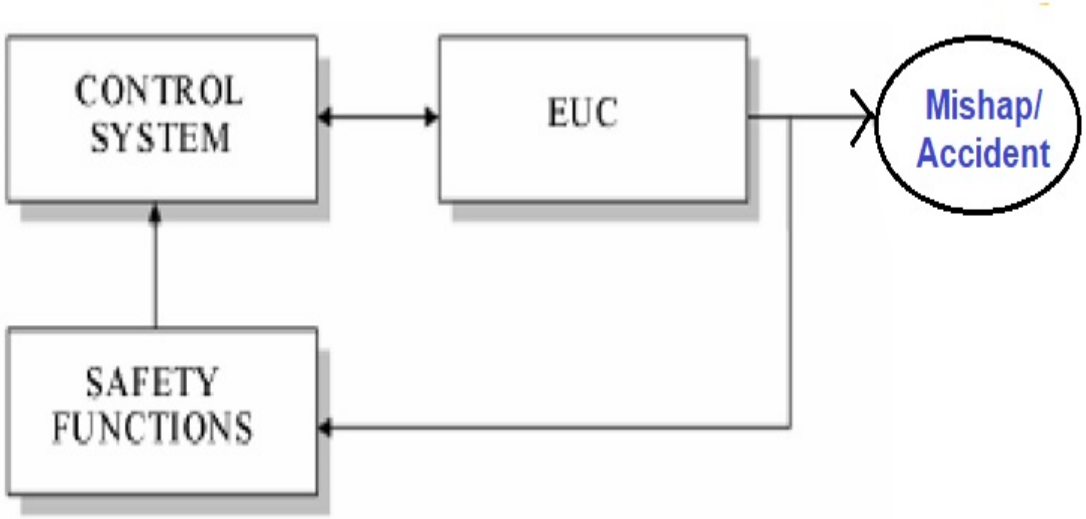
Definition according to IEC 61508:

"part of the overall safety relating to the EUC (Equipment under Control) and the EUC control system which depends on the correct functioning of the E/E/PE safety-related systems, other technology safety-related systems and external risk reduction facilities"



Functional Safety

- EUC - Equipment intended to provide a Function
  - i.e., equipment, machinery, apparatus or plant used for manufacturing, process, transportation, medical or other activities
- EUC control system - A system which controls the EUC and between them may pose a risk (it m*ay be integrated with the EUC, e.g., a microprocessor, or be remote from EUC).*
  - The threat/risk is commonly referred as a misdirected energy also accident or mishap



Functional Safety

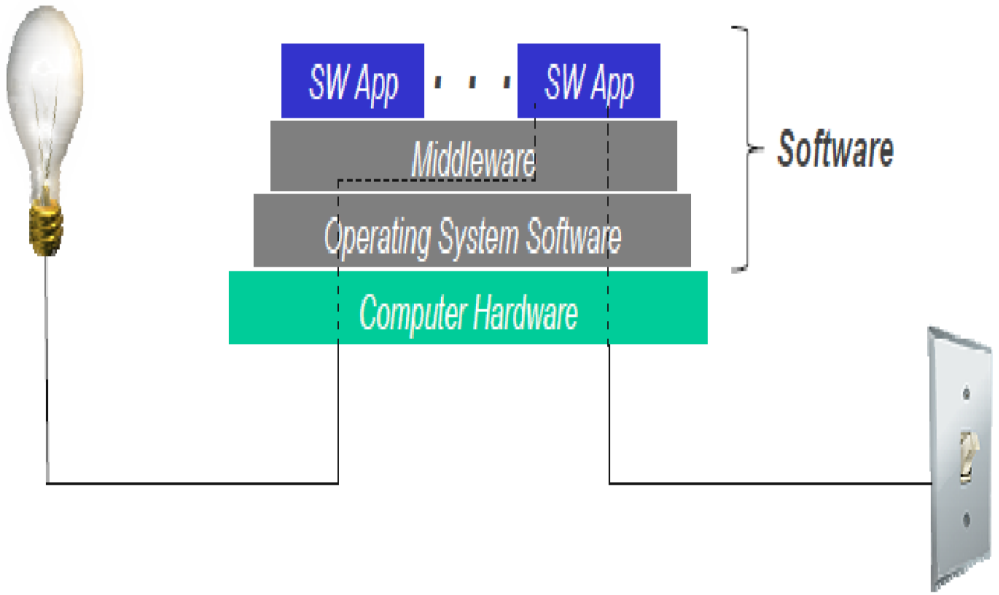
- Safety functions are to be provided to reduce the risks posed by the EUC and its control system
  - Safety functions may be provided in one or more protection systems as well as within the control system itself
- Any systems which are *designated to implement the required safety functions necessary to achieve a safe state for the EUC* are classified as safety-related systems

Importance of Functional Safety

- To protect against potential risks, including injuries and even death
- Product Certification (functional safety assessment)

Software Safety and Functional Safety

- Software Drives FS Requirements - IEC 61508-3
  - Electromechanical systems are rapidly being replaced by (software) programmable electronic systems due to:
    - Lower cost parts
    - Greater redesign flexibility
    - Ease of module reuse
    - Less PCB space required
    - Improved Efficiency
    - Greater functionality



Group Activities

- Based on an operational / software system of your choice identify any safety related systems

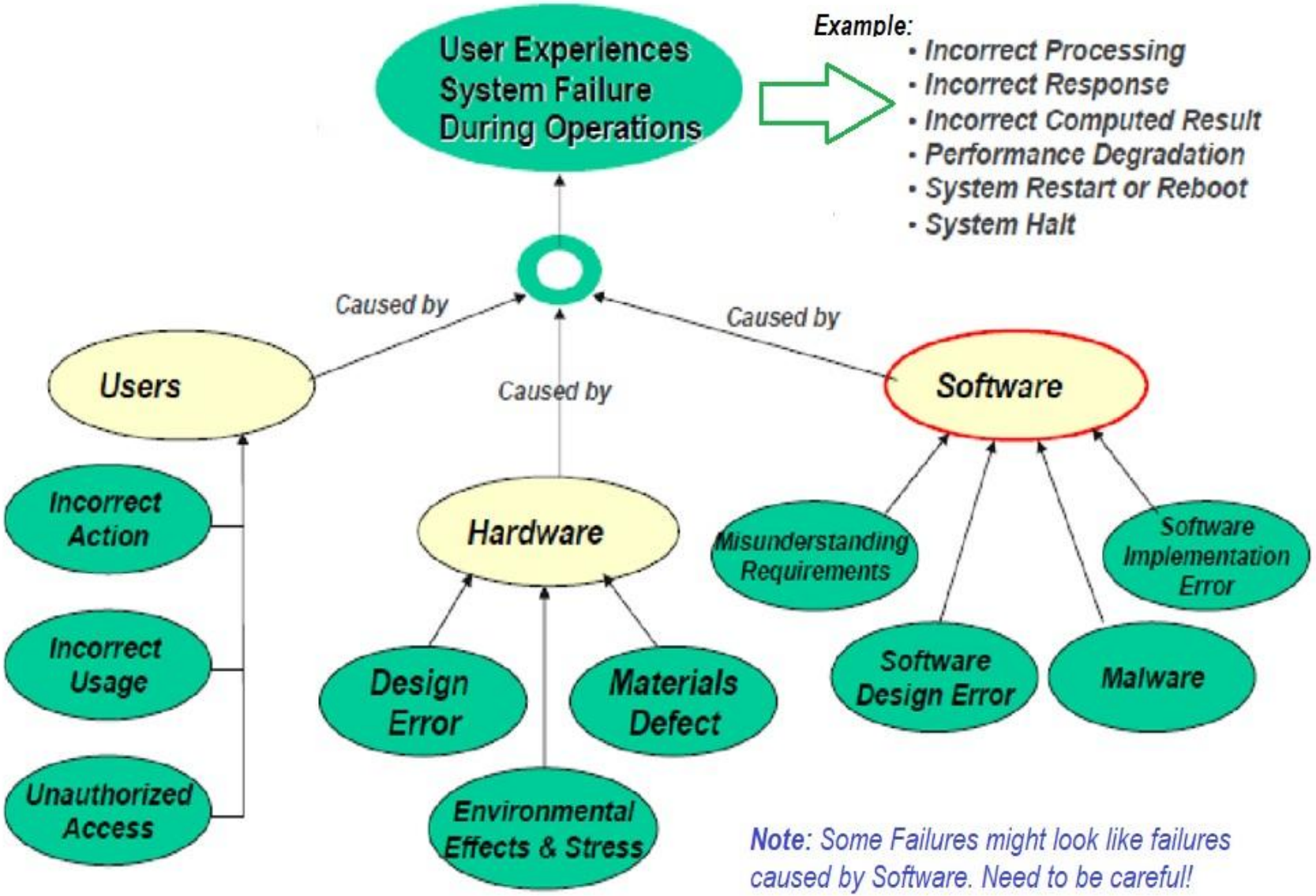
Key Terminologies

- Error - an incorrect step, process, or data definition
  - Example: an incorrect instruction in a computer program
- Defect - a condition or characteristic in hardware or software which is not in compliance with the specified configuration
  - A defect may exist without leading to a failure
- Fault/bug - any change in the state of a system or component which is considered abnormal and may deserve some type of corrective action
  - Example: software exceptions (e.g., divide by zero and file not found);
  - Faults are preliminary indications that a failure may have occurred

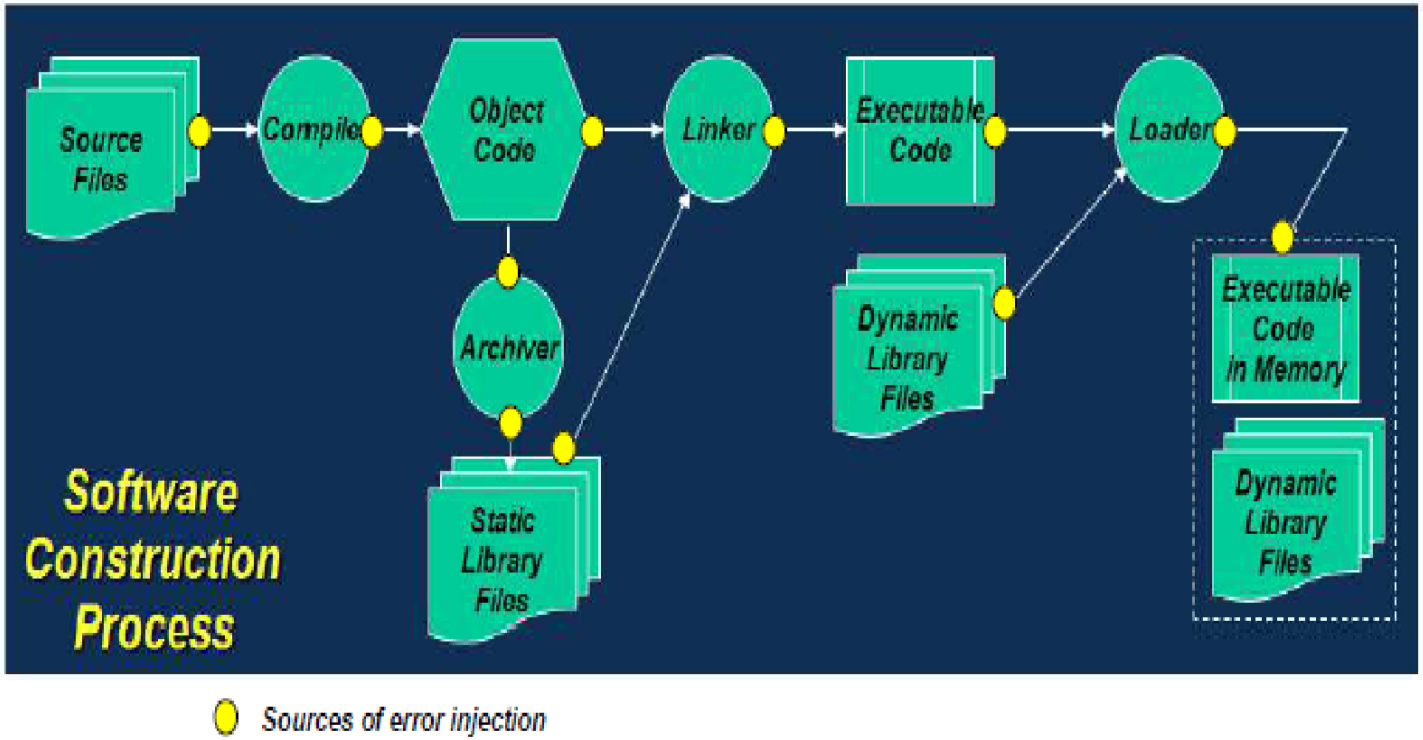
Key terminologies

- Failure - the inability of a system or component to perform its required functions within specified performance requirements
  - A fault may or may not lead to a failure.
  - One or more faults can become a failure, i.e. failures are the result of one or more faults.
- Fault tolerance - the ability of the system to withstand an unwanted event and maintain a safe and operational condition.
  - Determined by the number of faults that can occur in a system or subsystem without the occurrence of a failure.
- Creating failure tolerance requires a system-wide approach to the software and hardware design, so that a failure does not lead to an unsafe state

General Causes of Failure



Error Ejection in Software Construction

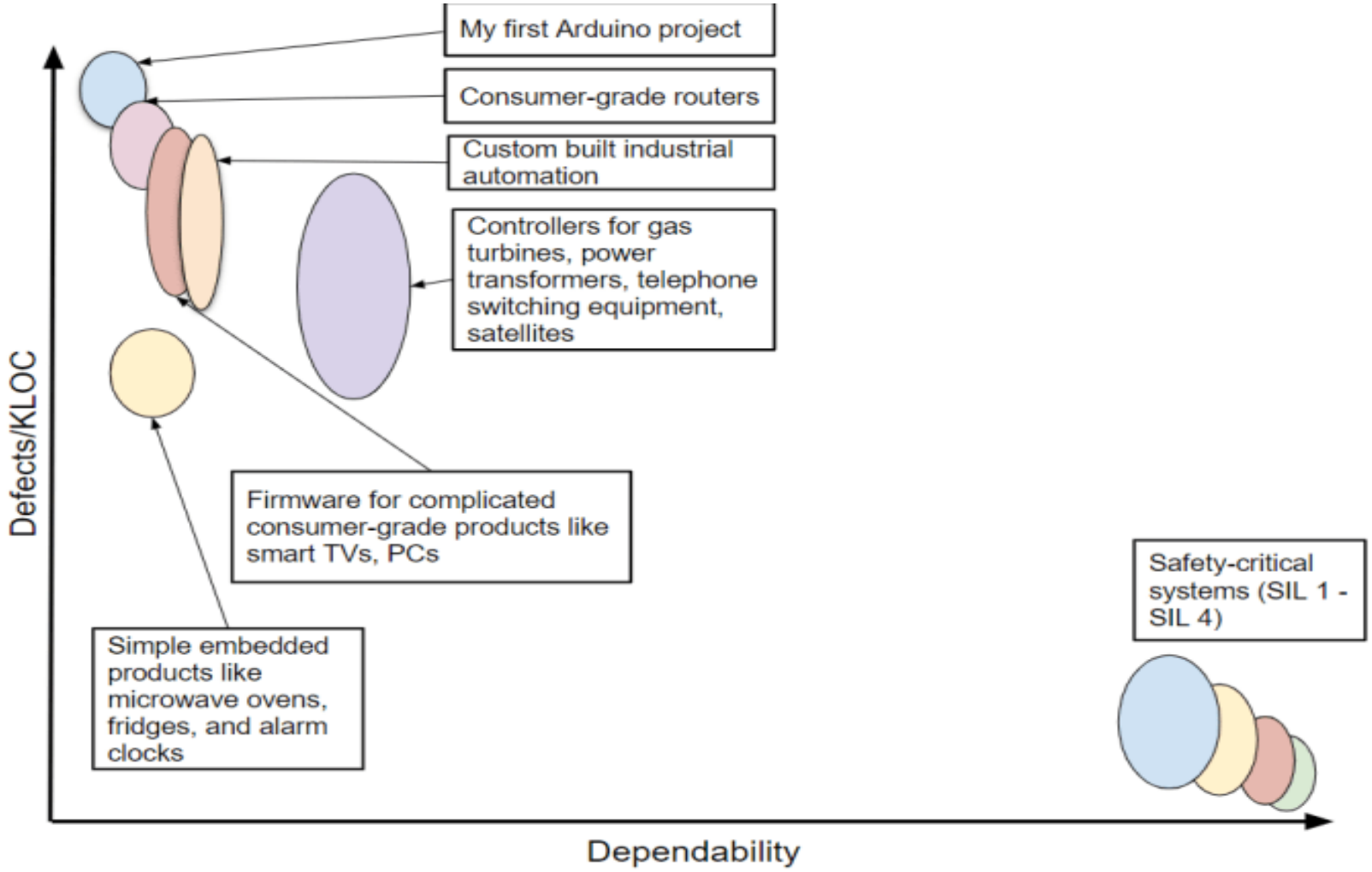


- Underlying defects in the source code, library files and in tools affecting code construction
- Environmental conditions operational software is not programmed to handle

Software Safety and Dependability

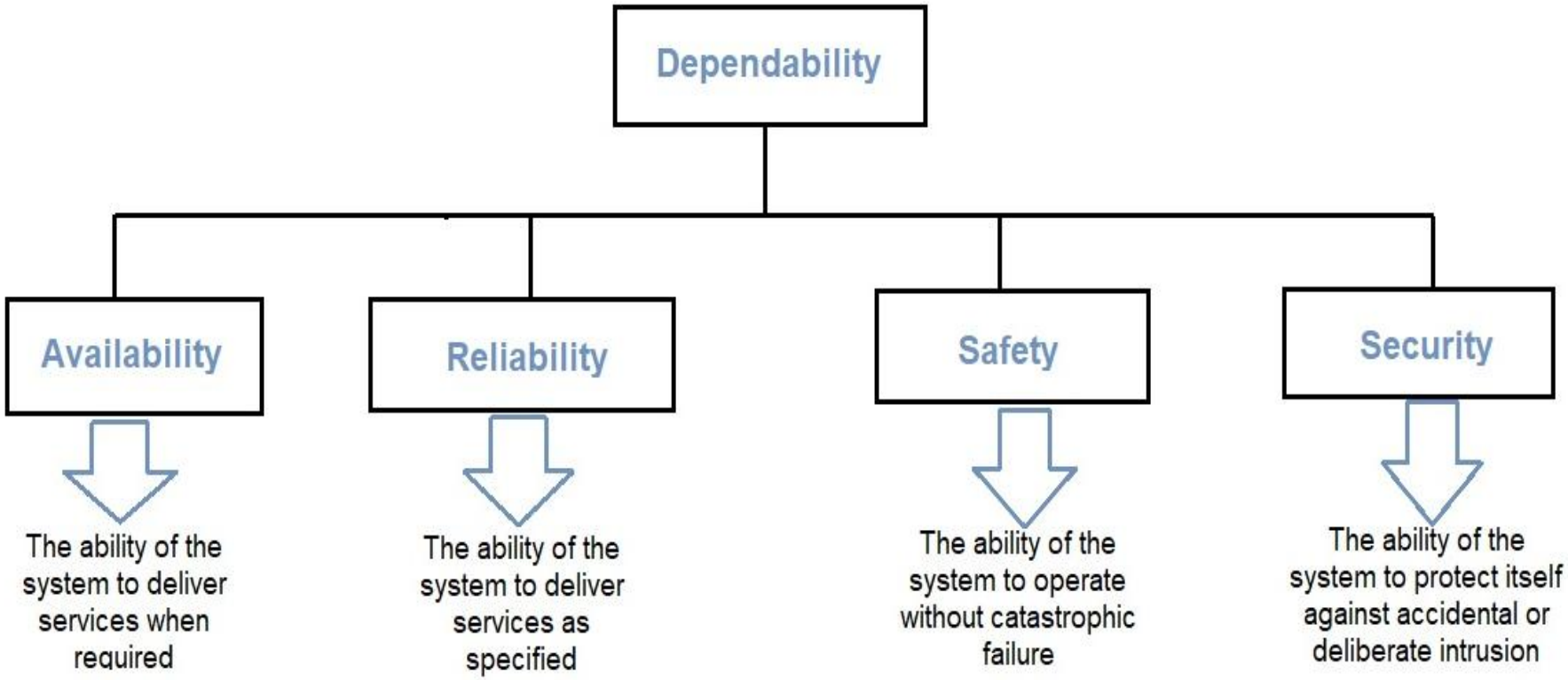
- Software developed and certified as safety-critical are highly dependable
  - The allowable failure rate for the most critical systems is strictly low
  - Safety-critical software should be designed, built and tested to ensure it has extremely low defect rates and extremely high dependability

Purported Dependability Index



Software Safety and Dependability..

- Dependability covers the related systems quality attributes of availability, reliability, safety and security, that are all interdependent:



Group Activities

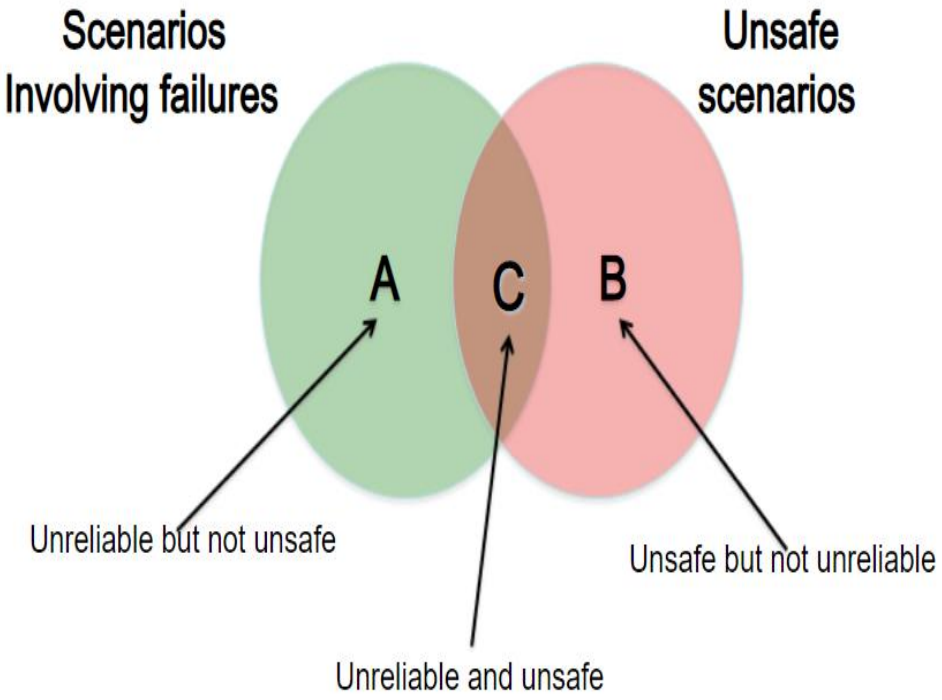
- Does
  1. Secure Software = Safe Software?
  2. Reliable Software/Lower software defect rates = Safe Software?
  3. Available software = Safe Software?

Safety vs Security

- Insecure software cannot be considered safe!
  - Safety standards are silence on aspects of addressing security
  - Security researchers have found glaring security vulnerabilities in many classes of safety-critical products
  - Because the entire safety case relies on the software behaving exactly as specified, it is a major problem if someone can make the system misbehave
  - Example:
    - Power grid attack in Ukraine -2015/2016
    - Autonomous vehicles: https://hackernoon.com/how-to-hack-self-driving-cars-vulnerabilities-in-autonomous-vehicles-jh3r37cz
    - Ransomware

Safety vs Reliability

- Functional failures closely relates to reliability attribute!
- Preventing Component or functional failures is NOT enough!



A: Car fails to start (Fail-Safe)
B: Mars Polar Lander
C: Ariene-5 Accident

Safety vs Reliability

- Software may be highly reliable and correct but be still unsafe when:
  - The software correctly implement the requirements but the specified behavior is unsafe from a system perspective
    - Case of Mars Polar Lander: https://en.wikipedia.org/wiki/Mars_Polar_Lander
  - The software requirement do not specify some particular behavior required for system safety (i.e., they are incomplete)
    - Case of Boeing 737 Max: https://studycorgi.com/case-of-boeing-737-max-safety-issues-and-management/
  - The software has unintended (and unsafe) behavior beyond what is specified in the requirements
    - Case of F-18 flight Loss
- Reliability requirements are concerned with making a system failure free, whereas safety requirements are concerned with making it mishap free
  - Safety and reliability may imply conflicting requirements, and thus the system cannot be built to maximize both

Safety vs Availability

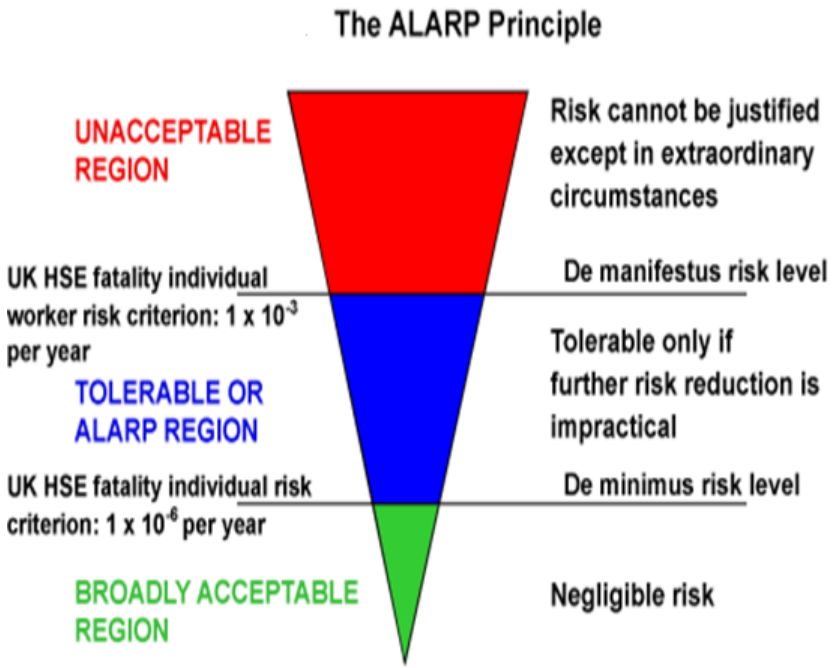
- Availability is the degree to which a system or component is operational and accessible when required for use
- Causes that lead to system unavailability include
  - Data corruption
  - Denial of service attacks
  - Hardware error
  - Mistake in software specification
  - Operational failure by human users
- Unavailability of a system can cause a safety accident
  - Fire alarm system (remotely monitored)

Group Activities

- Identify the influence of each of the following to the system availability
  - **Safety violation**
  - **Security violation**
  - **Reliability violation**

Goal of Software Safety

- System Safety integration
  - Reduce risk of serious hazards caused by/induced by software to acceptable levels
    - Risk is a combination of two things: the chance that the hazard will cause harm and how serious that harm could be
  - As Low As Reasonably Practicable (ALARP): Efforts to reduce risk should be continued until the incremental sacrifice in doing so is grossly disproportionate to the value of the incremental risk reduction achieved



The triangle represents the decreasing risk and the diminishing proportional benefit as risk is reduced.

Summary

- Safety is a systems quality attribute
  - Software engineering and software are contributors to safe systems and safe operations
- Safety engineering conducts system-wide hazard analysis
  - Software engineering works with Safety engineering to help identify and characterize hazards involving the command, control, and/or monitoring of critical functions necessary for safe operation of system
- Risk Consequences of Software Safety involve
  - People
  - Money
  - Environment

Appendix: Software Caused Incidents

- **Software Glitch Delayed Release of Results**
  - *(Sept. 8, '04, Las Vegas, NV) -- For the second time during a busy election, the county's election department is plagued with problems. The Registrar of Voters says that software mishap was just one problem they had to deal with and it must be fixed before the general election.*

- **Ford Recalls F150 and Lincoln Mark LT trucks for Brake Errors (2006)**
  - *Ford and Lincoln Mercury are recalling over 211,000 2006 Ford F-150 and 2006 Lincoln Mark LT trucks because a software glitch can disable the ABS brake warning system light if the system becomes inoperative.*

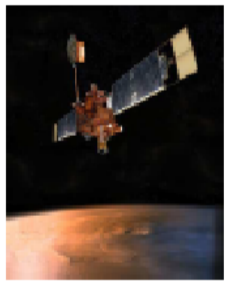
- **Prius Problems Traced to Software Glitch**

  - (May 18, 2005) *--Toyota Motor Corp is focusing efforts on a software problem in the popular hybrid Prius automobile after complaints that the gas-electric hybrid cars stall or shut down without warning while driving at highway speeds (2004 and 2005 model cars).*

- **Nissan Leaf recalled for SW**
  - *2011 -- Nissan Motor Co. is recalling 5,300 Leaf electric cars back to dealerships to fix a software glitch that can keep it from starting. Reprogram engine controller.*

Appendix: Software Caused Incidents

- **Mars Global Surveyer**
  - *Initiating event - two bad addresses uploaded while in orbit*
    - Software exceeded limits, locking solar panel gimbals
    - As designed, MGS reoriented itself to face panels toward sun
    - Battery on sun side overheated
    - Power management software design 'assumed' that battery overheating was due to overcharging and commanded charging system shutdown
  - *Vehicle was lost*



1996

- **Mars Polar Lander**
  - *On final descent, landing strut deployed as planned caused sensor vibrations*
    - Software misinterpreted vibration-induced signals from accelerometers as touchdown
  - *Software subsequently shut down the descent engine about 40 meters above the Martian surface*
  - *Hard landing, vehicle lost*

1999

- **Mars Climate Orbitor**
  - *During transit to Mars, units discrepancy (lb-secs vs. newton-secs) in software undiscovered*
    - Data was loaded into tables in units different from software input expectation
  - *In-transit trajectory errors accumulated, putting the vehicle too close to planet for orbit insertion burn*
  - *Vehicle lost*

1999

Appendix: Software Caused Incidents

- Mishaps where software-related problems were reported to play a role . . .

Year	Deaths	Description
1985	3	Therac-25 Software Design Flaw lead to radiation overdoses in treatment of cancer patients
1991	28	Software prevents patriot missile battery from targeting SCUD missile. Hits army barracks
1995	159	AA jet crashes into mountain in Cali, Columbia. Software presented insufficient and conflicting information to pilots who got lost
1997	1	Software causes morphine pump to deliver lethal dose to patient
2001	5	Crash of V-22 Osprey tilt-rotor helicopter caused by software anomaly
2003	3	Software failure contributes to power outage across NW U.S. and Canada

Questions

End of segment